

Praktikum 04

UX, RecyclerView, Fragment, dan Navigasi

Dosen pengampu: Muhayat, M.IT

Pada praktikum ini, Anda membangun WargaHub P04 dengan satu Activity dan tiga Fragment: Beranda, Daftar Laporan, dan Rincian Laporan. Daftar menampilkan 15 laporan fiktif menggunakan RecyclerView. Ketika satu laporan dipilih, aplikasi membuka rincian sesuai ID. Anda juga menguji daftar kosong, tombol Kembali, rotasi, dan aksesibilitas dasar.

Ikuti tahap A–J dari penyiapan lingkungan, pembuatan folder proyek baru, pengaturan dependensi, penulisan XML/Kotlin, hingga pengujian dan pengumpulan. P04 memakai data contoh lokal yang dibentuk dari kode; fitur database dan CRUD belum menjadi bagian latihan ini.

1. Identitas praktikum dan Sub-CPMK

Komponen	Keterangan
Mata kuliah	Praktikum Pemrograman Mobile • PTI23043 • 1 SKS
Semester dan pertemuan	Semester V • Pertemuan 4 • Praktikum 04
Tingkat dan kerja	Dasar hingga menengah • individual; diskusi uji berpasangan
Platform	Native Android • Kotlin • XML Views • ViewBinding
Sub-CPMK utama	Sub-CPMK 2: merancang dan menerapkan UI/UX yang responsif serta mudah digunakan; mengikuti arah acuan P03.
Fokus kompetensi	Daftar laporan, Fragment, argumen navigasi, back stack, serta evaluasi UX.
Prasyarat	P02: Activity, Intent, lifecycle. P03: XML, resources, binding, UI responsif.
Proyek latihan	WargaHubP04 • id.antasari.p04.wargahub
Luaran	Proyek tiga layar, daftar dan rincian, uji UX, screenshot, README, laporan refleksi.

Identitas mata kuliah menggunakan kode praktikum PTI23043 dan 1 SKS. Acuan P03 mencantumkan mata kuliah teori; struktur sepuluh bagian dan alur praktiknya dipertahankan untuk modul ini.



2. Tujuan dan indikator keberhasilan

Kemampuan	Indikator terukur
Menyiapkan proyek Views	Folder baru, package benar, Gradle Sync berhasil, template dapat Run.
Merancang UX tiga layar	Aksi utama, judul layar, serta hasil navigasi dapat dijelaskan.
Menampilkan daftar	RecyclerView memuat 15 data fiktif; daftar dapat digulir.
Menggunakan Adapter dan ViewHolder	Setiap baris menampilkan judul, kategori, serta status yang sesuai.
Mengelola Fragment	Binding dibuat saat View tersedia dan dilepas pada onDestroyView.
Mengirim argumen navigasi	ID bertipe Long membawa pilihan daftar ke rincian yang benar.
Mengelola kondisi UI	Daftar kosong mempunyai petunjuk dan aksi pemulihan.
Memeriksa hasil	Uji navigasi, rotasi, font besar, serta TalkBack dicatat dengan bukti.

Kriteria selesai: aplikasi dapat dibuka; Beranda → Daftar → Rincian bekerja; setiap pilihan menampilkan data yang benar; kembali tidak menambah layar duplikat; daftar kosong dapat dipulihkan; dan tidak ada crash pada skenario wajib. Hasil aktual wajib diisi setelah aplikasi dijalankan, bukan hanya berdasarkan Preview.

2.1. Alokasi kegiatan yang disarankan

Kegiatan	Alokasi usulan	Fokus
Orientasi dan demonstrasi	30 menit	Alur tugas, RecyclerView, Fragment, dan navigasi.
Praktik terbimbing	120 menit	Lingkungan, proyek, resources, dan implementasi inti.
Penyelesaian mandiri	120–180 menit	Melengkapi kode, memperbaiki error, serta menjalankan aplikasi.
Pengujian dan refleksi	45–60 menit	Uji tugas, bukti layar, README, dan laporan.

Alokasi merupakan perkiraan untuk latihan bertahap dan dapat dibagi ke beberapa sesi. Sesuaikan dengan jadwal kelas serta kondisi perangkat; bukan ketentuan beban waktu SKS.



3. Prasyarat dan persiapan perangkat

3.1. Lingkungan yang diperlukan

- Android Studio dengan dukungan Kotlin, Android SDK, dan emulator atau perangkat Android.
- SDK Platform API 36 sebagai konfigurasi acuan; compileSdk tidak boleh di bawah 35 untuk RecyclerView 1.4.0.
- Gradle JDK bawaan Android Studio atau JDK yang cocok dengan Android Gradle Plugin proyek.
- Akses internet untuk unduhan SDK, Gradle, dan dependensi pada penyiapan pertama.

3.2. Pemeriksaan sebelum membuat proyek

1. Buka Android Studio. Pada Welcome pilih More Actions → SDK Manager, atau gunakan Settings → Android SDK pada proyek yang terbuka. Nama menu dapat berbeda antarversi.
2. Pada SDK Platforms pastikan Android API 36 terpasang. Pada SDK Tools pastikan Platform-Tools dan Android Emulator tersedia jika memakai emulator.
3. Periksa Settings → Build, Execution, Deployment → Build Tools → Gradle. Gunakan Gradle JDK bawaan yang kompatibel; jangan mengganti versi JDK/Gradle secara acak.
4. Buka Device Manager. Buat emulator ponsel yang mendukung minSdk 26, lalu jalankan sampai layar utama Android tampil. Jika memakai HP, aktifkan USB debugging dan izinkan komputer.
5. Siapkan ruang kerja baru. Folder P03 tetap disimpan untuk perbandingan; jangan menjadikan folder proyek P03 sebagai lokasi P04.

Fitur P04 berjalan menggunakan data lokal dan tidak membutuhkan akun cloud atau permission INTERNET. Akses jaringan pada penyiapan awal dipakai alat pengembangan untuk mengunduh komponen, bukan oleh fitur aplikasi.

3.3. Perangkat uji dan data latihan

Sediakan uji portrait dan landscape, ukuran font standar dan diperbesar sekitar 1,3×–1,5×, serta TalkBack jika tersedia. Catat model perangkat atau nama emulator, versi Android, dan versi aplikasi. Seluruh laporan memakai data fiktif; status Simulasi bukan status tiket layanan nyata.



4. Kasus alur pengguna dan contoh hasil

Seorang warga ingin memeriksa daftar contoh masalah lingkungan. Ia membuka WargaHub, memilih Lihat Daftar Laporan, menggulir daftar, lalu memilih satu laporan. Aplikasi menampilkan judul, kategori, status contoh, dan deskripsi yang sama dengan pilihan tersebut. Pengguna dapat kembali ke daftar melalui tombol aplikasi atau navigasi sistem.

Langkah pengguna	Hasil yang diharapkan
Buka aplikasi	Beranda dan penjelasan data contoh tampil.
Pilih Lihat Daftar Laporan	Terlihat daftar 15 laporan dan jumlah data.
Gulir dan pilih zona latihan tertentu	Rincian sesuai judul serta ID laporan terpilih.
Pilih Kembali ke Daftar	Daftar sebelumnya terbuka; tidak dibuat daftar baru.
Pilih Uji Daftar Kosong	Jumlah menjadi 0; pesan kosong dan aksi pemulihan tampil.
Pilih Tampilkan Data Contoh	Kelima belas laporan contoh muncul kembali.
Putar perangkat	Tujuan navigasi dan mode kosong tetap sesuai pada rekreasi biasa.

Alur tugas utama: Beranda → Daftar → Rincian → kembali ke Daftar → kembali ke Beranda. Tombol uji kosong hanya mengubah data yang ditampilkan oleh Adapter; sumber data contoh tidak dihapus. Tidak ada formulir tambah laporan pada proyek P04 ini. Formulir P03 tetap dipelajari dari proyek sebelumnya dan dapat dikembangkan bersama penyimpanan pada tahap berikutnya.

5. Ringkasan konsep dan analogi

5.1. UX dan keadaan tampilan

UX adalah pengalaman pengguna ketika mencapai tujuan. Dalam P04, kualitasnya tampak dari kemudahan menemukan daftar, memahami baris yang dipilih, membaca rincian, dan kembali tanpa kebingungan. Judul layar berasal dari navigation graph. Pesan daftar kosong menjelaskan keadaan serta menawarkan tindakan berikutnya.

5.2. RecyclerView, Adapter dan ViewHolder

RecyclerView menampilkan kumpulan data dengan memakai ulang View baris. Analoginya, beberapa map tampilan digunakan bergantian untuk menampilkan kartu laporan yang sedang terlihat. **Adapter**



menghubungkan data dengan baris, **ViewHolder** memegang referensi View baris, dan **LinearLayoutManager** mengatur susunan vertikal (Android Developers, n.d.-a).

Komponen	Tanggung jawab dalam P04
Laporan	Model data satu laporan dengan ID unik.
LaporanContoh	Menyediakan data fiktif tetap untuk latihan.
item_laporan.xml	Rancangan satu baris yang dapat dipilih.
LaporanAdapter	Membuat dan mengisi baris menggunakan ViewBinding.
ListAdapter dan DiffUtil	Mengelola daftar dan membandingkan identitas serta isi item.
LinearLayoutManager	Mengatur urutan dan gulir vertikal daftar.

RecyclerView tidak dibungkus ScrollView karena ia sudah mengelola gulir. Beranda dan rincian tetap memakai ScrollView untuk teks yang bisa bertambah. Pada layar daftar, ScrollView hanya dipakai sebagai tampilan alternatif pesan kosong; bukan sebagai parent RecyclerView.

5.3. Activity Fragment dan lifecycle View

MainActivity menjadi wadah utama dan mengatur toolbar. NavHostFragment menampilkan HomeFragment, ListFragment, atau DetailFragment sesuai navigation graph. Fragment adalah bagian UI di dalam Activity, bukan Activity baru. Fragment tidak didaftarkan sebagai activity di Manifest.

View milik Fragment dapat dihancurkan saat Fragment berada di back stack. Karena itu _binding bersifat nullable, diisi pada onViewCreated, dan dilepas pada onDestroyView. Jangan memakai binding setelah View dihancurkan (Android Developers, n.d.-b).

5.4. Navigation graph argumen dan back stack

Navigation graph adalah peta tujuan. NavController menjalankan perpindahan dan mengelola riwayat navigasi. Aplikasi mengirim ID kecil bertipe Long, lalu mencari rincian dari sumber yang sama. Pendekatan ini menghindari pengiriman seluruh objek antarhalaman (Android Developers, n.d.-c).

Kembali ke daftar menggunakan navigateUp, bukan navigate menuju ListFragment lagi. Toolbar memakai NavigationUI agar judul dan ikon Up mengikuti tujuan aktif (Android Developers, n.d.-d). Pada alur sederhana tanpa deep link ini, Up dan Back mengarah ke tujuan sebelumnya; keduanya tidak selalu identik pada semua rancangan aplikasi.



5.5. Sintaks Kotlin yang digunakan

Sintaks	Makna sederhana
data class dan val	Model data dengan nilai properti yang tidak diubah langsung.
object	Satu objek sumber data contoh.
(1..15).map { ... }	Membentuk 15 objek dari rentang angka.
find { it.id == id }	Mencari data berdasarkan ID; hasil bisa null.
submitList(data)	Mengirim daftar baru ke ListAdapter.
bundleOf(key to value)	Menyusun argumen navigasi di dalam Bundle.
_binding dan requireNotNull	Membatasi akses binding pada saat View masih tersedia.
savedInstanceState	Menyimpan flag kecil untuk rekreasi; bukan database.

6. Langkah kerja dan file yang dikerjakan

Kerjakan A–J berurutan. Blok berlabel kode lengkap mengganti seluruh file yang disebut. Potongan Gradle hanya digabungkan ke blok yang sudah ada. Teks penjelasan, nomor halaman, dan label kode tidak ikut disalin ke editor. Jalankan template pada tahap A; jalankan aplikasi akhir setelah E–H lengkap agar tidak terhenti oleh referensi yang belum dibuat.

6.A. Menginisiasi lingkungan dan membuat folder proyek

1. Pastikan pemeriksaan lingkungan pada bagian 3 selesai dan emulator/HP dikenali Android Studio.
2. Buka File Explorer. Buat folder induk C:\AndroidProjects, lalu buat folder pengumpulan P04_NIM_Nama di dalamnya. Ganti NIM dan Nama sesuai identitas mahasiswa.
3. Di dalam P04_NIM_Nama, buat folder screenshots dan docs. Lokasi proyek yang akan dibuat adalah C:\AndroidProjects\P04_NIM_Nama\WargaHubP04. Folder WargaHubP04 akan diisi Android Studio.
4. Pada Android Studio pilih New Project → Phone and Tablet → Empty Views Activity. Jangan memilih Empty Activity yang menggunakan Compose.
5. Isi Name: WargaHubP04; Package name: id.antasari.p04.wargahub; Save location: lokasi proyek baru tadi; Language: Kotlin; Minimum SDK: API 26. Gunakan Kotlin DSL jika pilihan konfigurasi build ditampilkan.



6. Klik Finish. Tunggu Gradle Sync selesai. Jika muncul instalasi SDK yang diperlukan, selesaikan melalui SDK Manager.
7. Pilih perangkat dari pemilih device, lalu tekan Run. Pastikan aplikasi template terbuka sebelum mengubah kode.

Checkpoint A: folder baru tersedia; template Views dapat dibuka di emulator/HP; package dan lokasi proyek benar.

6.B. Menyiapkan Gradle ViewBinding dan package

1. Buka app/build.gradle.kts melalui Gradle Scripts. Pastikan ini milik Module :app, bukan file Gradle pada root proyek.
2. Di dalam android, pastikan namespace dan applicationId bernilai id.antasari.p04.wargahub; gunakan compileSdk 36 dan minSdk 26. Pertahankan targetSdk, plugins, dan konfigurasi Java/Kotlin dari template yang kompatibel.
3. Aktifkan viewBinding dalam buildFeatures. Tambahkan dependensi berikut di dependencies yang sudah ada. Pertahankan AppCompat, Core KTX, dan Activity dari template.
4. Jika dependensi telah dikelola melalui libs.versions.toml, gunakan satu deklarasi untuk setiap library. Jangan menambah alias dan string koordinat untuk artifact yang sama.
5. Tekan Sync Now. Jika sync gagal, baca pesan paling awal pada Build Output dan perbaiki jaringan, SDK, atau kompatibilitas toolchain terlebih dahulu.

Potongan app build gradle kts yang digabungkan

```
android {
    // Pertahankan konfigurasi lain milik template.
    compileSdk = 36
    buildFeatures { viewBinding = true }
    defaultConfig { minSdk = 26 }
}

dependencies {
    // Pertahankan Core KTX, Activity, dan AppCompat dari template.
    implementation("androidx.recyclerview:recyclerview:1.4.0")
    implementation("androidx.fragment:fragment-ktx:1.8.9")
    implementation("androidx.navigation:navigation-fragment-ktx:2.9.5")
    implementation("androidx.navigation:navigation-ui-ktx:2.9.5")
}
```



Versi di atas adalah versi acuan tetap untuk latihan, bukan klaim versi terbaru. RecyclerView 1.4.0 membutuhkan compileSdk setidaknya 35; modul memilih 36. Gunakan Android Studio dan AGP yang mendukung SDK tersebut. Jangan menurunkan compileSdk hanya untuk menghilangkan pesan dependensi (Android Developers, n.d.-e, n.d.-f, n.d.-g).

1. Pada panel Project, gunakan tampilan Android atau Project. Buka app/src/main/java/id/antasari/p04/wargahub. Klik kanan package utama → New → Package; buat data, adapter, dan ui.
2. Pada res, siapkan direktori layout, values, dan navigation. Untuk navigation: klik kanan res → New → Android Resource Directory → Resource type: navigation.
3. Buat Kotlin Class/File dengan tipe File atau Class sesuai nama di bawah; lalu ganti seluruh isinya dengan kode modul. Hindari wizard Fragment yang menambahkan kode lain jika belum dipahami.

Lokasi relatif dari app src main	File yang disiapkan
java/id/antasari/p04/wargahub/	MainActivity.kt
java/id/antasari/p04/wargahub/data/	Laporan.kt dan LaporanContoh.kt
java/id/antasari/p04/wargahub/adapter/	LaporanAdapter.kt
java/id/antasari/p04/wargahub/ui/	HomeFragment.kt, ListFragment.kt, DetailFragment.kt
res/layout/	activity_main.xml, fragment_home.xml, fragment_list.xml, item_laporan.xml, fragment_detail.xml
res/values/	strings.xml, colors.xml, dimens.xml, styles.xml, themes.xml
res/navigation/	nav_graph.xml
res/values-night/ jika tersedia	themes.xml dengan isi tema yang sama pada tahap D
Direktori app/src/main/	AndroidManifest.xml

Checkpoint B: *sync berhasil; ViewBinding aktif; package dan direktori resource tersedia. Binding layout baru muncul setelah XML dibuat.*

6.C. Membuat teks warna ukuran dan gaya kontrol

1. Buka res/values/strings.xml lalu ganti isinya dengan kode lengkap. Nama key harus sama dengan yang dipakai XML dan Kotlin.
2. Ganti colors.xml dengan empat warna berikut; tema lama akan diganti pada tahap D agar tidak lagi membutuhkan warna template.



3. Buat dimens.xml dan styles.xml melalui New → Values Resource File. Setiap file hanya mempunyai satu root resources.
4. Style Text menyiapkan ukuran, warna, dan dimensi yang dipakai ulang. Title mewarisi Text; Button mewarisi gaya tombol AppCompat. Pemakaian style mengurangi pengulangan atribut pada lima layout.

Kode lengkap strings.xml

app/src/main/res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">WargaHub P04</string>
    <string name="nav_home">Beranda</string>
    <string name="nav_list">Daftar Laporan</string>
    <string name="nav_detail">Rincian Laporan</string>
    <string name="home_title">WargaHub</string>
    <string name="home_intro">Pelajari cara membaca laporan lingkungan melalui daftar dan rincian.</string>
    <string name="demo_note">Data contoh untuk latihan. Bukan laporan nyata dan belum memakai database.</string>
    <string name="open_list">Lihat Daftar Laporan</string>
    <string name="list_help">Pilih satu laporan untuk melihat rinciannya.</string>
    <string name="try_empty">Uji Daftar Kosong</string>
    <string name="show_sample">Tampilkan Data Contoh</string>
    <string name="empty_message">Belum ada laporan pada tampilan latihan. Pilih Tampilkan Data Contoh.</string>
    <string name="list_count">%1$d laporan contoh</string>
    <string name="row_status">Status contoh: %1$s</string>
    <string name="row_description">%1$s. Kategori %2$s. Status contoh %3$s. Buka rincian.</string>
    <string name="detail_help">Periksa kesesuaian rincian dengan laporan yang dipilih.</string>
    <string name="detail_category">Kategori: %1$s</string>
    <string name="detail_status">Status contoh: %1$s</string>
    <string name="detail_missing">Laporan tidak ditemukan. Kembali ke daftar dan pilih data contoh.</string>
    <string name="back_list">Kembali ke Daftar</string>
</resources>
```

Kode lengkap colors.xml

app/src/main/res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="p04_navy">#10254A</color>
    <color name="p04_slate">#334155</color>
    <color name="p04_page">#F4FAFC</color>
    <color name="p04_white">#FFFFFF</color>
</resources>
```



Kode lengkap dimens.xml

app/src/main/res/values/dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="page_padding">16dp</dimen>
    <dimen name="item_gap">12dp</dimen>
</resources>
```

Kode lengkap styles.xml

app/src/main/res/values/styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="Widget.WargaHubP04.Text" parent="">
        <item name="android:layout_width">match_parent</item>
        <item name="android:layout_height">wrap_content</item>
        <item name="android:textSize">16sp</item>
        <item name="android:textColor">@color/p04_slate</item>
        <item name="android:layout_marginBottom">12dp</item>
    </style>
    <style name="Widget.WargaHubP04.Title"
        parent="Widget.WargaHubP04.Text">
        <item name="android:textSize">24sp</item>
        <item name="android:textStyle">bold</item>
        <item name="android:textColor">@color/p04_navy</item>
    </style>
    <style name="Widget.WargaHubP04.Button"
        parent="Widget.AppCompat.Button">
        <item name="android:layout_width">match_parent</item>
        <item name="android:layout_height">wrap_content</item>
        <item name="android:minHeight">48dp</item>
        <item name="android:textAllCaps">false</item>
        <item name="android:textSize">16sp</item>
        <item name="android:textColor">@color/p04_white</item>
        <item name="backgroundTint">@color/p04_navy</item>
    </style>
</resources>
```

Parameter %1$d dipakai untuk jumlah data. Parameter %1$s, %2$s, dan %3$s dipakai untuk teks dinamis. Jangan menggantinya dengan nilai contoh di dalam resource. Warna status tidak digunakan sebagai satu-satunya petunjuk; kata Simulasi selalu ditampilkan.

Checkpoint C: *resources dapat dibaca; tidak ada key ganda; satuan teks sp dan jarak dp tersusun konsisten*.



6.D. Menyiapkan tema dan AndroidManifest

1. Ganti res/values/themes.xml dengan isi berikut. Tema NoActionBar diperlukan karena aplikasi menyediakan Toolbar sendiri.
2. Jika res/values-night/themes.xml tersedia, ganti isinya dengan isi yang sama persis. Modul memakai tema terang tetap agar warna dan kontras konsisten pada latihan; tema gelap menjadi pengayaan.
3. Buka AndroidManifest.xml, lalu ganti seluruh isinya dengan kode lengkap. Hanya MainActivity didaftarkan sebagai Activity launcher. Fragment didefinisikan pada navigation graph, bukan Manifest.

Kode lengkap themes.xml

app/src/main/res/values/themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="Theme.WargaHubP04"
        parent="Theme.AppCompat.Light.NoActionBar">
        <item name="colorPrimary">@color/p04_navy</item>
        <item name="colorPrimaryDark">@color/p04_navy</item>
        <item name="colorAccent">@color/p04_navy</item>
        <item name="android:windowLightStatusBar">true</item>
        <item name="android:windowBackground">@color/p04_page</item>
    </style>
</resources>
```

Salin kembali XML tema di atas ke app/src/main/res/values-night/themes.xml apabila direktori night ada. Jangan membiarkan tema template yang sama namanya mempunyai konfigurasi berbeda pada mode malam.

Kode lengkap AndroidManifest.xml

app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/Theme.WargaHubP04">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
```




```
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Manifest ini sengaja minimal dan tidak menambahkan izin jaringan, lokasi, kamera, atau penyimpanan. Ikon launcher bawaan boleh tetap berada di folder mipmap meskipun tidak dirujuk oleh Manifest minimal ini.

Checkpoint D: *satu application dan satu MainActivity launcher; tema terang konsisten; tidak ada deklarasi Fragment sebagai Activity.*

6.E. Membuat wadah utama dan lima layout

1. Buka activity_main.xml dan pilih Code. Ganti dengan layout wadah berikut.
2. Buat empat layout baru melalui klik kanan res/layout → New → Layout Resource File: fragment_home, fragment_list, item_laporan, dan fragment_detail.
3. Isi setiap file dengan kode lengkap. Jangan memberi akhiran .xml pada nama resource jika dialog membuatkannya otomatis.
4. Perhatikan FragmentContainerView mengisi sisa tinggi setelah toolbar menggunakan layout_weight. Di LinearLayout, 0dp dengan weight mempunyai arti berbeda dari match constraints pada ConstraintLayout P03.
5. Pada tahap E–H, beberapa referensi bisa tampak merah sebelum file tujuan dibuat. Selesaikan rangkaian, lalu Sync/Build; jangan menghapus ID hanya untuk menghilangkan penanda sementara.

Kode lengkap activity_main.xml

app/src/main/res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/mainRoot"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/p04_page">
    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```




```
        android:minHeight="?attr/actionBarSize"
        android:background="@color/p04_white" />
    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/navHostFragment"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph" />
</LinearLayout>
```

Wadah hanya menerapkan insets sekali melalui MainActivity pada tahap H. Fragment tidak perlu menambahkan padding system bars lagi. app:defaultNavHost membantu NavHost menerima Back sistem.

Kode lengkap fragment_home.xml

app/src/main/res/layout/fragment_home.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="@dimen/page_padding">
        <TextView
            style="@style/Widget.WargaHubP04.Title"
            android:text="@string/home_title" />
        <TextView
            style="@style/Widget.WargaHubP04.Text"
            android:text="@string/home_intro" />
        <TextView
            style="@style/Widget.WargaHubP04.Text"
            android:text="@string/demo_note" />
        <Button
            android:id="@+id/btnLihatLaporan"
            style="@style/Widget.WargaHubP04.Button"
            android:text="@string/open_list" />
    </LinearLayout>
</ScrollView>
```



Kode lengkap fragment_list.xml

app/src/main/res/layout/fragment_list.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="@dimen/page_padding">
    <TextView
        style="@style/Widget.WargaHubP04.Text"
        android:text="@string/list_help" />
    <TextView
        android:id="@+id/tvJumlah"
        style="@style/Widget.WargaHubP04.Text"
        android:accessibilityLiveRegion="polite" />
    <Button
        android:id="@+id/btnModeKosong"
        style="@style/Widget.WargaHubP04.Button"
        android:text="@string/try_empty" />
    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">
        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/rvLaporan"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:clipToPadding="false"
            android:paddingBottom="@dimen/item_gap" />
        <ScrollView
            android:id="@+id/emptyContainer"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:fillViewport="true"
            android:visibility="gone">
            <TextView
                style="@style/Widget.WargaHubP04.Text"
                android:paddingTop="16dp"
                android:text="@string/empty_message" />
        </ScrollView>
    </FrameLayout>
</LinearLayout>
```

RecyclerView dan emptyContainer adalah dua anak FrameLayout yang tampil bergantian. Tombol Uji Daftar Kosong tetap tersedia ketika data berjumlah 0 sehingga pengguna dapat memulihkan tampilan tanpa membuka ulang aplikasi.



Kode lengkap item_laporan.xml

app/src/main/res/layout/item_laporan.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="@dimen/item_gap"
    android:orientation="vertical"
    android:minHeight="72dp"
    android:padding="16dp"
    android:clickable="true"
    android:focusable="true"
    android:background="?attr/selectableItemBackground">
    <TextView
        android:id="@+id/tvJudul"
        style="@style/Widget.WargaHubP04.Text"
        android:textStyle="bold"
        android:importantForAccessibility="no" />
    <TextView
        android:id="@+id/tvKategori"
        style="@style/Widget.WargaHubP04.Text"
        android:importantForAccessibility="no" />
    <TextView
        android:id="@+id/tvStatus"
        style="@style/Widget.WargaHubP04.Text"
        android:importantForAccessibility="no" />
</LinearLayout>
```

Satu baris menjadi satu target interaksi. Deskripsi aksesibilitas baris diisi oleh Adapter; tiga teks di dalamnya tidak dibaca berulang sebagai target terpisah. Uji pembaca layar pada perangkat karena keluaran dapat berbeda menurut layanan dan versi Android.

Kode lengkap fragment_detail.xml

app/src/main/res/layout/fragment_detail.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="@dimen/page_padding">
        <TextView
```



```
            style="@style/Widget.WargaHubP04.Text"
            android:text="@string/detail_help" />
        <TextView
            android:id="@+id/tvJudul"
            style="@style/Widget.WargaHubP04.Title" />
        <TextView
            android:id="@+id/tvKategori"
            style="@style/Widget.WargaHubP04.Text" />
        <TextView
            android:id="@+id/tvStatus"
            style="@style/Widget.WargaHubP04.Text" />
        <TextView
            android:id="@+id/tvDeskripsi"
            style="@style/Widget.WargaHubP04.Text" />
        <TextView
            style="@style/Widget.WargaHubP04.Text"
            android:text="@string/demo_note" />
        <Button
            android:id="@+id/btnKembaliDaftar"
            style="@style/Widget.WargaHubP04.Button"
            android:text="@string/back_list" />
    </LinearLayout>
</ScrollView>
```

Checkpoint E: *lima file layout tersedia; ID sesuai modul; daftar mengisi ruang yang tersisa; beranda dan rincian memiliki gulir sendiri.*

6.F. Membuat model data sumber contoh dan Adapter

1. Pada package data, buat Laporan.kt. ID menggunakan Long agar identitas item tidak bergantung pada urutan baris.
2. Buat LaporanContoh.kt pada package data. Rentang 1..15 membentuk 15 laporan dengan ID unik dan isi yang dapat dibentuk ulang.
3. Pada package adapter, buat LaporanAdapter.kt. Perhatikan import Laporan dan ItemLaporanBinding.
4. Pelajari urutan Adapter: buat ViewHolder, isi View dengan data, lalu panggil callback ketika baris dipilih. Jangan menyimpan position sebagai identitas laporan.

Kode lengkap Laporan.kt

app/src/main/java/id/antasari/p04/wargahub/data/Laporan.kt

```
package id.antasari.p04.wargahub.data

data class Laporan(
```



```
    val id: Long,
    val judul: String,
    val kategori: String,
    val deskripsi: String,
    val status: String
)
```

Kode lengkap LaporanContoh.kt

app/src/main/java/id/antasari/p04/wargahub/data/LaporanContoh.kt

```
package id.antasari.p04.wargahub.data

object LaporanContoh {
    private val kategori = listOf(
        "Lampu jalan padam", "Sampah menumpuk", "Drainase tersumbat"
    )

    val semua: List<Laporan> = (1..15).map { nomor ->
        val jenis = kategori[(nomor - 1) % kategori.size]
        Laporan(
            id = nomor.toLong(),
            judul = "$jenis di zona latihan $nomor",
            kategori = jenis,
            deskripsi = "Contoh masalah di zona latihan $nomor. " +
                "Data fiktif ini dipakai untuk mempelajari daftar " +
                "dan navigasi rincian laporan.",
            status = "Simulasi"
        )
    }

    fun cari(id: Long): Laporan? = semua.find { it.id == id }
}
```

Literal data contoh berada di Kotlin karena merupakan isi dataset latihan, sedangkan label antarmuka tetap di strings.xml. Ketika proses aplikasi dibuat ulang, objek ini membentuk data contoh yang sama. Hal itu bukan bukti penyimpanan laporan pengguna.

Kode lengkap LaporanAdapter.kt

app/src/main/java/id/antasari/p04/wargahub/adapter/LaporanAdapter.kt

```
package id.antasari.p04.wargahub.adapter

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.DiffUtil
```



```
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import id.antasari.p04.wargahub.R
import id.antasari.p04.wargahub.data.Laporan
import id.antasari.p04.wargahub.databinding.ItemLaporanBinding

class LaporanAdapter(
    private val onItemClick: (Laporan) -> Unit
) : ListAdapter<Laporan, LaporanAdapter.LaporanHolder>(Pembanding) {

    class LaporanHolder(
        private val binding: ItemLaporanBinding
    ) : RecyclerView.ViewHolder(binding.root) {
        fun bind(item: Laporan, onClick: (Laporan) -> Unit) {
            val context = binding.root.context
            binding.tvJudul.text = item.judul
            binding.tvKategori.text = item.kategori
            binding.tvStatus.text = context.getString(
                R.string.row_status, item.status
            )
            binding.root.contentDescription = context.getString(
                R.string.row_description,
                item.judul, item.kategori, item.status
            )
            binding.root.setOnClickListener { onClick(item) }
        }
    }

    override fun onCreateViewHolder(
        parent: ViewGroup, viewType: Int
    ): LaporanHolder {
        val binding = ItemLaporanBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
        )
        return LaporanHolder(binding)
    }

    override fun onBindViewHolder(holder: LaporanHolder, position: Int) {
        holder.bind(getItem(position), onItemClick)
    }

    companion object {
        private val Pembanding = object : DiffUtil.ItemCallback<Laporan>() {
            override fun areItemsTheSame(a: Laporan, b: Laporan) =
```




```
                a.id == b.id
            override fun areContentsTheSame(a: Laporan, b: Laporan) =
                a == b
        }
    }
}
```

areItemsTheSame membandingkan ID, sedangkan areContentsTheSame membandingkan isi data class. bind memperbarui seluruh teks dan callback untuk item yang sedang ditampilkan. Lambda menerima objek laporan yang diikat, sehingga pilihan tetap mengacu pada ID yang benar setelah View digunakan ulang.

Checkpoint F: *model, sumber data, dan Adapter tersedia; ID 1–15 unik; binding item cocok dengan item_laporan.xml.*

6.G. Membuat tiga Fragment dan logika tampilannya

1. Pada package ui, buat HomeFragment.kt, ListFragment.kt, dan DetailFragment.kt. Gunakan androidx.fragment.app.Fragment, bukan android.app.Fragment.
2. Konstruktor Fragment(R.layout...) menyerahkan pembuatan View kepada framework. Isi binding pada onViewCreated menggunakan bind(view).
3. HomeFragment membuka daftar. ListFragment memasang LayoutManager dan Adapter serta mengirim ID. DetailFragment mengambil ID lalu mencari data yang sesuai.
4. Pada setiap Fragment, lepaskan binding pada onDestroyView. ListFragment juga melepas Adapter dari RecyclerView agar referensi View dan callback tidak tertahan.

Kode lengkap HomeFragment.kt

app/src/main/java/id/antasari/p04/wargahub/ui/HomeFragment.kt

```
package id.antasari.p04.wargahub.ui

import android.os.Bundle
import android.view.View
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import id.antasari.p04.wargahub.R
import id.antasari.p04.wargahub.databinding.FragmentHomeBinding

class HomeFragment : Fragment(R.layout.fragment_home) {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = requireNotNull(_binding)

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
```




```
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentHomeBinding.bind(view)
        binding.btnLihatLaporan.setOnClickListener {
            val nav = findNavController()
            if (nav.currentDestination?.id == R.id.homeFragment) {
                nav.navigate(R.id.action_home_to_list)
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Kode lengkap ListFragment.kt

app/src/main/java/id/antasari/p04/wargahub/ui/ListFragment.kt

```
package id.antasari.p04.wargahub.ui

import android.os.Bundle
import android.view.View
import androidx.core.os.bundleOf
import androidx.core.view.isVisible
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import id.antasari.p04.wargahub.R
import id.antasari.p04.wargahub.adapter.LaporanAdapter
import id.antasari.p04.wargahub.data.LaporanContoh
import id.antasari.p04.wargahub.databinding.FragmentListBinding

class ListFragment : Fragment(R.layout.fragment_list) {
    private var _binding: FragmentListBinding? = null
    private val binding get() = requireNotNull(_binding)
    private var contohKosong = false
    private var laporanAdapter: LaporanAdapter? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        contohKosong = savedInstanceState?.getBoolean(KEY_KOSONG) ?: false
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentListBinding.bind(view)
```



```
        val adapter = LaporanAdapter { laporan ->
            val nav = findNavController()
            if (nav.currentDestination?.id == R.id.listFragment) {
                nav.navigate(
                    R.id.action_list_to_detail,
                    bundleOf(DetailFragment.ARG_ID to laporan.id)
                )
            }
        }
        adapter.stateRestorationPolicy =
            RecyclerView.Adapter.StateRestorationPolicy.PREVENT_WHEN_EMPTY
        laporanAdapter = adapter
        binding.rvLaporan.layoutManager = LinearLayoutManager(requireContext())
        binding.rvLaporan.adapter = adapter
        binding.btnModeKosong.setOnClickListener {
            contohKosong = !contohKosong
            tampilkanDaftar()
        }
        tampilkanDaftar()
    }

    private fun tampilkanDaftar() {
        val data = if (contohKosong) emptyList() else LaporanContoh.semua
        laporanAdapter?.submitList(data)
        binding.rvLaporan.isVisible = data.isNotEmpty()
        binding.emptyContainer.isVisible = data.isEmpty()
        binding.tvJumlah.text = getString(R.string.list_count, data.size)
        binding.btnModeKosong.setText(
            if (contohKosong) R.string.show_sample else R.string.try_empty
        )
    }

    override fun onSaveInstanceState(outState: Bundle) {
        outState.putBoolean(KEY_KOSONG, contohKosong)
        super.onSaveInstanceState(outState)
    }

    override fun onDestroyView() {
        binding.rvLaporan.adapter = null
        laporanAdapter = null
        _binding = null
        super.onDestroyView()
    }

    companion object {
        private const val KEY_KOSONG = "contoh_kosong"
    }
}
```



contohKosong disimpan sebagai flag kecil dalam Bundle. submitList menerima daftar kosong atau sumber contoh; tidak menghapus isi LaporanContoh. PREVENT_WHEN_EMPTY membantu menunda pemulihan state RecyclerView sampai Adapter mempunyai data. Posisi gulir tetap perlu diperiksa pada uji nyata.

Pemeriksaan currentDestination menghindari pengulangan action dari tujuan yang sudah berganti akibat ketukan cepat. Pemeriksaan ini bukan pengganti logika idempotensi pengiriman jaringan; P04 tidak mengirim laporan ke server.

Kode lengkap DetailFragment.kt

app/src/main/java/id/antasari/p04/wargahub/ui/DetailFragment.kt

```
package id.antasari.p04.wargahub.ui

import android.os.Bundle
import android.view.View
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import id.antasari.p04.wargahub.R
import id.antasari.p04.wargahub.data.LaporanContoh
import id.antasari.p04.wargahub.databinding.FragmentDetailBinding

class DetailFragment : Fragment(R.layout.fragment_detail) {
    private var _binding: FragmentDetailBinding? = null
    private val binding get() = requireNotNull(_binding)

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentDetailBinding.bind(view)
        val id = arguments?.getLong(ARG_ID, -1L) ?: -1L
        val laporan = LaporanContoh.cari(id)
        if (laporan == null) {
            binding.tvJudul.setText(R.string.detail_missing)
            binding.tvKategori.visibility = View.GONE
            binding.tvStatus.visibility = View.GONE
            binding.tvDeskripsi.visibility = View.GONE
        } else {
            binding.tvJudul.text = laporan.judul
            binding.tvKategori.text = getString(
                R.string.detail_category, laporan.kategori
            )
            binding.tvStatus.text = getString(
                R.string.detail_status, laporan.status
            )
            binding.tvDeskripsi.text = laporan.deskripsi
        }
        binding.btnKembaliDaftar.setOnClickListener {
            findNavController().navigateUp()
        }
```



```
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }

    companion object {
        const val ARG_ID = "laporan_id"
    }
}
```

Ketika ID tidak ditemukan, rincian menampilkan pesan yang dapat dipahami dan tombol kembali tetap bekerja. Jangan memaksa laporan menjadi non-null memakai !! karena kesalahan argumen seharusnya ditangani sebagai keadaan UI yang aman.

Checkpoint G: *tiga Fragment memakai binding yang benar; ListFragment menyiapkan daftar; DetailFragment memiliki cabang data tidak ditemukan.*

6.H. Menyusun navigation graph dan MainActivity

1. Buat res/navigation/nav_graph.xml melalui New → Android Resource File pada direktori navigation. Isi seluruh file berikut.
2. Periksa android:name pada setiap fragment harus sama persis dengan nama package dan kelas Kotlin. startDestination adalah homeFragment.
3. Argumen laporan_id bertipe long dan mempunyai default -1L. Nilai ini sama dengan key ARG_ID pada DetailFragment.
4. Ganti MainActivity.kt dengan kode lengkap. NavController diambil dari NavHostFragment, kemudian dihubungkan dengan ActionBar yang memakai Toolbar.
5. Jangan menambahkan Fragment awal secara manual melalui FragmentTransaction dan jangan memanggil setGraph setiap rotasi. NavHost memulihkan navigasi yang sudah ada.

Kode lengkap nav_graph.xml

app/src/main/res/navigation/nav_graph.xml

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/homeFragment">
    <fragment
```



```
        android:id="@+id/homeFragment"
        android:name="id.antasari.p04.wargahub.ui.HomeFragment"
        android:label="@string/nav_home">
        <action
            android:id="@+id/action_home_to_list"
            app:destination="@id/listFragment" />
    </fragment>
    <fragment
        android:id="@+id/listFragment"
        android:name="id.antasari.p04.wargahub.ui.ListFragment"
        android:label="@string/nav_list">
        <action
            android:id="@+id/action_list_to_detail"
            app:destination="@id/detailFragment" />
    </fragment>
    <fragment
        android:id="@+id/detailFragment"
        android:name="id.antasari.p04.wargahub.ui.DetailFragment"
        android:label="@string/nav_detail">
        <argument
            android:name="laporan_id"
            app:argType="long"
            android:defaultValue="-1L" />
    </fragment>
</navigation>
```

Modul mengirim Bundle secara eksplisit agar mahasiswa dapat melihat hubungan key dan tipe data. Safe Args tidak diaktifkan; jangan menambahkan plugin atau mengimpor kelas Directions/Args yang tidak dibuat. Safe Args dapat dipelajari setelah alur dasar dipahami.

Kode lengkap MainActivity.kt

app/src/main/java/id/antasari/p04/wargahub/MainActivity.kt

```
package id.antasari.p04.wargahub

import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import androidx.navigation.NavController
import androidx.navigation.fragment.NavHostFragment
import androidx.navigation.ui.AppBarConfiguration
import androidx.navigation.ui.navigateUp
import androidx.navigation.ui.setupActionBarWithNavController
import id.antasari.p04.wargahub.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
```



```
    private lateinit var binding: ActivityMainBinding
    private lateinit var navController: NavController
    private lateinit var appBarConfiguration: AppBarConfiguration

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
        aturInsets()
        setSupportActionBar(binding.toolbar)
        val navHost = supportFragmentManager
            .findFragmentById(R.id.navHostFragment) as NavHostFragment
        navController = navHost.navController
        appBarConfiguration = AppBarConfiguration(navController.graph)
        setupActionBarWithNavController(navController, appBarConfiguration)
    }

    override fun onSupportNavigateUp(): Boolean {
        return navController.navigateUp(appBarConfiguration) ||
            super.onSupportNavigateUp()
    }

    private fun aturInsets() {
        ViewCompat.setOnApplyWindowInsetsListener(binding.mainRoot) {
            view, insets ->
            val bars = insets.getInsets(
                WindowInsetsCompat.Type.systemBars() or
                    WindowInsetsCompat.Type.displayCutout()
            )
            view.setPadding(bars.left, bars.top, bars.right, bars.bottom)
            insets
        }
        ViewCompat.requestApplyInsets(binding.mainRoot)
    }
}
```

Insets melindungi toolbar serta konten dari system bars dan display cutout. Padding ditetapkan dari nilai insets terbaru, bukan ditambahkan terus ke padding sebelumnya. P04 tidak memiliki field ketik, sehingga penanganan IME formulir P03 tidak diperlukan pada alur inti.

Checkpoint H: *graph memuat tiga tujuan dan dua action; label toolbar mengikuti layar; tombol Up serta Back dikelola oleh Navigation Component.*



6.I. Build menjalankan aplikasi dan memeriksa alur

1. Simpan semua file. Tekan Sync Project with Gradle Files, lalu jalankan Build → Make/Assemble Project sesuai menu yang tersedia. Baca Build Output sampai tidak ada error.
2. Pilih app dan perangkat uji, tekan Run. Tunggu instalasi dan pembukaan aplikasi selesai.
3. Pastikan Beranda muncul satu kali. Tekan Lihat Daftar Laporan; jumlah harus 15.
4. Gulir hingga zona latihan 10 atau setelahnya. Pilih item dan cocokkan judul, kategori, serta deskripsinya pada rincian.
5. Tekan Kembali ke Daftar. Ulangi pemilihan dua laporan lain; rincian tidak boleh tertukar.
6. Di daftar, tekan Uji Daftar Kosong. Pastikan jumlah 0, daftar tersembunyi, dan petunjuk pemulihan terlihat. Tekan Tampilkan Data Contoh; jumlah kembali 15.
7. Uji Up toolbar dan Back sistem dari rincian serta daftar. Pada Beranda, Back mengikuti perilaku keluar aplikasi yang lazim; jangan membuat Activity baru untuk kembali.
8. Putar perangkat pada rincian dan saat daftar kosong. Periksa judul layar, ID yang ditampilkan, serta mode kosong.

Alternatif pemeriksaan melalui Terminal Android Studio

```
# Windows PowerShell, jalankan dari root WargaHubP04
.\gradlew.bat assembleDebug

# macOS atau Linux
./gradlew assembleDebug
```

Terminal harus berada di folder yang memuat settings.gradle.kts dan gradlew. Gunakan wrapper bawaan proyek, bukan memasang Gradle global. APK debug yang berhasil dibangun berada di app/build/outputs/apk/debug/app-debug.apk. Pesan BUILD SUCCESSFUL adalah bukti build; keberhasilan UX tetap dibuktikan lewat uji perangkat.

Checkpoint I: *APK terbangun; tiga layar, daftar kosong, pemulihan data contoh, dan navigasi dapat dijalankan tanpa crash.*

6.J. Menyempurnakan UX dan menyiapkan bukti

1. Besarkan font perangkat sekitar 1,3×–1,5×. Buka ulang Beranda, daftar, dan rincian; pastikan teks serta tombol dapat dibaca.
2. Uji landscape dan ponsel kecil. Baris harus tetap dapat digulir dan tombol utama dapat dicapai.



3. Aktifkan TalkBack dan ikuti tutorial gesturenya. Telusuri daftar; dengarkan judul, kategori, status contoh, dan tujuan aksi tiap baris.
4. Minta satu teman menjalankan tugas “Temukan rincian laporan zona latihan 10, lalu kembali ke daftar” tanpa menunjukkan urutan ketukan.
5. Catat berhasil/gagal, waktu tugas, permintaan bantuan, dan satu alasan kemudahan atau kesulitan. Tetapkan awal/akhir pengukuran yang sama bila membandingkan perubahan.
6. Perbaiki satu masalah yang ditemukan. Uji ulang skenario yang sama, lalu simpan bukti sebelum–sesudah.
7. Tulis README dan laporan. Isi matriks hasil aktual serta nama screenshot; jangan menuliskan Lulus untuk skenario yang belum dijalankan.

Checkpoint J: *bukti navigasi, daftar kosong, rotasi, dan aksesibilitas dasar tersedia; satu temuan UX mempunyai catatan tindak lanjut.*

7. Checkpoint dan hasil yang diharapkan

Tahap	Bukti minimum
A–B	Folder baru, template Run, sinkronisasi dependensi berhasil.
C–D	Resources, tema, dan Manifest konsisten.
E	Lima layout; ID dan ruang gulir sesuai fungsinya.
F	Model, 15 ID unik, dan Adapter yang mengisi seluruh baris.
G–H	Tiga Fragment, navigation graph, argumen Long, dan toolbar terhubung.
I–J	APK dapat dijalankan; matriks uji, screenshot, serta refleksi selesai.

7.1. Bantuan ketika terjadi error

Gejala	Pemeriksaan dan tindakan
Unresolved reference Binding	Periksa viewBinding, nama XML, error resources pertama, lalu Build ulang.
AAPT resource not found	Cocokkan key string, warna, style, layout, dan ID. Pastikan root resources tunggal.
Missing SDK atau AAR metadata	Pasang SDK 36; pastikan AGP mendukungnya dan baca kebutuhan dependensi.



Gejala	Pemeriksaan dan tindakan
Could not find navigation action	Cocokkan action graph dengan listener dan currentDestination; jangan menggandakan navigasi.
Class not found Fragment	Periksa package ui, android:name pada graph, dan penggunaan AndroidX Fragment.
Daftar kosong tanpa alasan	Periksa LinearLayoutManager, adapter, submitList, tinggi RecyclerView, serta flag contohKosong.
Rincian tidak sesuai	Kirim laporan.id bertipe Long dan cocokkan key laporan_id; jangan mengirim position.
Crash saat kembali atau rotasi	Baca Logcat; periksa akses binding sesudah onDestroyView dan navigasi manual yang menduplikasi host.
Toolbar ganda atau tema crash	Pastikan tema NoActionBar dipakai pada values dan values-night; Toolbar hanya satu.

Untuk setiap masalah, catat pesan error paling awal, file/baris, perubahan yang dilakukan, dan hasil setelah diuji ulang. Jangan menghapus fungsi hanya agar build melewati pemeriksaan sementara.

8. Skenario pengujian normal dan gagal

Gunakan APK dan versi kode yang sama selama satu rangkaian uji. Catat perangkat, versi Android, orientasi, skala font, hasil diharapkan, hasil aktual, status, dan bukti. Tabel berikut berisi harapan, bukan hasil pengujian yang sudah terjadi.

ID dan kondisi	Aksi uji	Hasil yang diharapkan
T01 Beranda	Buka aplikasi dari launcher.	Satu Beranda; penjelasan data contoh terlihat.
T02 Daftar	Tekan Lihat Daftar Laporan.	Jumlah 15 dan baris berisi judul, kategori, status.
T03 Gulir	Gulir ke laporan zona latihan 10–15.	Daftar lancar dan item bawah dapat dipilih.
T04 Rincian	Pilih dua laporan berbeda secara bergantian.	Rincian sesuai ID masing-masing.
T05 Tombol kembali	Dari rincian tekan Kembali ke Daftar.	Kembali tanpa menambah daftar baru.
T06 Up toolbar	Tekan Up di rincian lalu di daftar.	Rincian → Daftar → Beranda.
T07 Back sistem	Ulangi alur lalu gunakan Back sistem.	Urutan kembali masuk akal; tidak crash.



ID dan kondisi	Aksi uji	Hasil yang diharapkan
T08 Kosong	Tekan Uji Daftar Kosong.	Jumlah 0; pesan dan pemulihan terlihat.
T09 Pulihkan	Tekan Tampilkan Data Contoh.	Jumlah menjadi 15; baris kembali tampil.
T10 Rotasi rincian	Putar perangkat saat rincian terbuka.	Layar rincian dan data ID tetap sesuai.
T11 Rotasi kosong	Aktifkan mode kosong lalu putar.	Mode kosong dan label tombol tetap sesuai.
T12 Font dan landscape	Perbesar font; uji ketiga layar dalam landscape.	Teks dapat dibaca, daftar dan rincian dapat digulir.
T13 TalkBack	Telusuri baris lalu buka rincian.	Nama baris bermakna; aksi utama dapat dijalankan.
T14 Ketukan cepat	Ketuk aksi dan item cepat berulang.	Tidak crash atau membuat layar duplikat.
T15 Mulai ulang	Tutup proses aplikasi lalu buka dari launcher.	Data contoh tetap tersedia saat daftar dibuka; bukan bukti database.

8.1. Uji cabang laporan tidak ditemukan

1. Untuk uji terkontrol sementara, pada ListFragment ganti nilai bundleOf dari laporan.id menjadi -99L. Jangan mengubah definisi graph.
2. Build dan Run ulang, buka daftar lalu pilih satu baris. Rincian harus menampilkan pesan tidak ditemukan dan tombol kembali tetap bekerja.
3. Kembalikan nilai argumen menjadi laporan.id, lalu Build/Run lagi. Pastikan rincian normal kembali sesuai. Kode akhir yang dikumpulkan menggunakan ID asli.

Beri ID T16 untuk uji ini. Screenshot pesan kesalahan berasal dari versi uji sementara; jelaskan perubahan tersebut di laporan agar tidak disalahartikan sebagai perilaku normal.

8.2. Format catatan hasil aktual

Kolom catatan	Isi yang harus ditulis
Identitas uji	ID skenario, tanggal, versi aplikasi atau commit.
Lingkungan	Perangkat/emulator, versi Android, orientasi, font, TalkBack.
Pelaksanaan	Langkah singkat dan hasil diharapkan.
Hasil aktual	Apa yang benar-benar terlihat atau terdengar.



Kolom catatan	Isi yang harus ditulis
Status dan bukti	Lulus/Perlu Perbaikan/Belum Diuji; nama screenshot atau catatan.
Tindak lanjut	Dugaan penyebab, perubahan, serta hasil uji ulang.

Untuk uji tugas teman, definisikan berhasil sebagai rincian zona latihan 10 yang tepat terbuka tanpa bantuan. Waktu dimulai saat Beranda siap dan diakhiri saat rincian benar terlihat. Tanyakan kemudahan pada skala 1–5 disertai alasan. Ini instrumen latihan sederhana, bukan SUS atau bukti sertifikasi usability.

9. Tugas inti remedial dan pengayaan

9.1. Tugas inti yang wajib diselesaikan

1. Buat WargaHubP04 pada folder baru dan gunakan package yang konsisten dengan modul.
2. Buat sketsa Beranda, Daftar, dan Rincian serta jelaskan aksi maju dan kembali.
3. Implementasikan seluruh kode inti, 15 laporan fiktif, RecyclerView, tiga Fragment, dan navigation graph.
4. Pastikan daftar kosong dapat dipulihkan dan argumen ID salah ditangani dengan pesan yang aman.
5. Jalankan T01–T12 dan T14–T16. Upayakan T13; jika layanan tidak tersedia, tulis keterbatasan dan rencana uji perangkat alternatif.
6. Lakukan satu uji tugas dengan teman dan tulis satu temuan serta perbaikan yang diperiksa ulang.

9.2. Remedial untuk tingkat dasar

Mulai dari memastikan template dapat Run, lalu periksa satu kelompok file setiap kali. Cocokkan nama layout dengan Binding, package dengan import, dan ID graph dengan listener. Gunakan Build Output untuk memilih satu error pertama, bukan mengubah banyak file sekaligus. Selesaikan alur tiga layar sebelum menambah dekorasi.

9.3. Pengayaan tingkat menengah

- Tambahkan pencarian judul secara lokal dan bedakan “tidak ada laporan” dengan “tidak ada hasil pencarian”.
- Tambahkan penanda laporan pilihan tanpa mengubah ID data dan periksa pemulihan state yang relevan.



- Gunakan Safe Args pada salinan atau branch latihan setelah dependensi dan plugin disiapkan dengan benar.
- Perbaiki batas lebar konten pada tablet atau tema gelap dengan tetap menguji kontras dan keterbacaan.

Pengayaan tidak menggantikan tugas inti. Jangan menambahkan CRUD atau menyatakan data persisten sebelum mekanisme penyimpanan benar-benar dibangun serta diuji.

9.4. Pertanyaan refleksi

- Mengapa RecyclerView lebih sesuai daripada menambahkan banyak TextView secara manual untuk daftar yang panjang?
- Apa perbedaan tanggung jawab Adapter, ViewHolder, dan LayoutManager?
- Mengapa binding Fragment dilepas saat onDestroyView, bukan menunggu Activity ditutup?
- Mengapa argumen berisi ID lebih tepat daripada posisi baris?
- Apa risiko jika tombol Kembali justru menjalankan navigate ke daftar baru?
- Apa perbedaan data contoh yang muncul lagi setelah restart dengan penyimpanan laporan permanen?
- Perubahan UX apa yang paling membantu teman saat mengerjakan tugas, dan apa buktinya?

10. Rubrik format pengumpulan dan referensi

10.1. Rubrik individual P04

Aspek	Bobot	Indikator
Kesesuaian fungsi	40%	Daftar dan rincian tepat, navigasi kembali benar, kondisi kosong dan gagal tertangani.
Implementasi kode dan UI	25%	Adapter, binding lifecycle, graph, resources, serta layout terbaca dan konsisten.
Pengujian dan perbaikan	25%	Hasil aktual, bukti skenario wajib, uji tugas, dan tindak lanjut temuan.
Penjelasan dan refleksi	10%	README, sketsa, penjelasan konsep, dan batas data simulasi.

Bobot ini merupakan usulan penilaian praktikum individual. Gunakan ketentuan dosen untuk rekap nilai mata kuliah dan penjadwalan pengumpulan.



10.2. Format pengumpulan

Kumpulkan folder P04_NIM_Nama yang dibuat pada tahap A. Di dalamnya ada WargaHubP04 sebagai root proyek Android, README.md, docs/Laporan_P04.docx atau PDF, dan screenshots. Tidak perlu membuat proyek di dalam folder proyek lain.

Bagian	Isi minimum
WargaHubP04/	Source project, Gradle Wrapper, settings, build scripts, dan version catalog bila dipakai.
README.md	Nama, NIM, kelas, perangkat, versi alat, cara membuka dan Run, ringkasan fitur serta batas data contoh.
docs/Laporan_P04	Sketsa, penjelasan Adapter/Fragment/navigasi, matriks uji, uji tugas teman, temuan dan refleksi.
screenshots/	Minimal 6 gambar: Beranda, Daftar, Rincian, Kosong, Landscape/font besar, dan ID tidak ditemukan.

Jika memakai ZIP, sertakan gradlew, gradlew.bat, serta gradle/wrapper agar proyek dapat dibangun ulang. Keluarkan folder build, .gradle, caches, dan .idea yang tidak diperlukan; keluarkan local.properties yang berisi lokasi SDK komputer serta keystore/kredensial pribadi. Jangan mengecualikan folder gradle/wrapper hanya karena namanya mirip cache .gradle.

10.3. Checklist sebelum mengumpulkan

- Aplikasi dapat dibuka dan menampilkan alur Beranda → Daftar → Rincian.
- Laporan yang dipilih cocok dengan ID, judul, kategori, status, dan deskripsi.
- Daftar kosong mempunyai pesan serta pemulihan; ID salah mempunyai pesan aman.
- Tombol Kembali, Up, Back sistem, dan rotasi telah diperiksa tanpa layar duplikat.
- Font besar serta landscape diuji; temuan TalkBack atau keterbatasannya dicatat.
- Binding dilepas dengan benar dan tidak ada error build yang diketahui.
- Kode argumen uji -99L telah dikembalikan menjadi laporan.id.
- README, laporan, matriks aktual, dan screenshot dapat dibuka.



10.4. Referensi teknis

Dokumentasi diperiksa pada 5 November 2026. Modul menggunakan jalur Views/Fragment untuk kesinambungan P02–P03, meskipun sebagian dokumentasi Android juga menyediakan pendekatan Compose.

Android Developers. (n.d.-a). Create dynamic lists with RecyclerView.

https://developer.android.com/develop/ui/views/layout/recyclerview

Android Developers. (n.d.-b). View binding.

https://developer.android.com/topic/libraries/view-binding

Android Developers. (n.d.-c). Pass data between destinations.

https://developer.android.com/guide/navigation/use-graph/pass-data

Android Developers. (n.d.-d). Connect UI components to NavController using NavigationUI.

https://developer.android.com/guide/navigation/integrations/ui

Android Developers. (n.d.-e). RecyclerView release notes.

https://developer.android.com/jetpack/androidx/releases/recyclerview

Android Developers. (n.d.-f). Fragment release notes.

https://developer.android.com/jetpack/androidx/releases/fragment

Android Developers. (n.d.-g). Navigation release notes.

https://developer.android.com/jetpack/androidx/releases/navigation

Muhayat. (2026). Modul Praktikum 03 UI responsif dan formulir laporan [Modul pembelajaran]. Program Studi Teknologi Informasi, UIN Antasari Banjarmasin.

Muhayat. (2026). Kuliah 05 Interaksi pengguna dan standar usability dalam aplikasi mobile [Bahan perkuliahan]. Program Studi Teknologi Informasi, UIN Antasari Banjarmasin.